

# BIM and ReadSeal: Safe Early Reuse of Shared GPU Buffers Across Processes

Xingbo Hui, Xuetao Jiang, Guodong Ye, Ruochen Shao, Tianyi Wang, Sicheng Wu, Rui Zhou\*, Qingguo Zhou\*  
School of Information Science and Engineering, Lanzhou University, Lanzhou, China

\*Co-corresponding authors: zr@lzu.edu.cn, zhouqg@lzu.edu.cn

**Abstract**—Processes in perception pipelines share large GPU tensors to avoid per-recipient copies. When may the producer overwrite a shared buffer with the next frame? CUDA stream-only release waits for submitted GPU work and can miss a reader that has not started. Full retention waits for every result and can leave the GPU idle while new frames are refused. Our key insight separates two lifetimes: results retain a frame’s identity until delivery, but its storage is needed only until the last read completes. BEV-in-Memory (BIM) protects pending readers through per-recipient borrows. ReadSeal analyzes storage sharing within each model to find the last-read boundary and checks that the resulting plan matches execution. Computation on private intermediate results can then overlap reuse of the shared buffer. In an A100 pipeline with four recipients and early source reads, one BIM slot delivers  $1.31\text{--}1.50\times$  the correct, timely results of one-slot full retention at 25–40 Hz, with 8–11 ms more mean latency and 15–19 ms more per-run 95th-percentile latency. ReadSeal preserves the delivery benefit of hand-placed boundaries while re-deriving them after model changes without manual boundary annotations. Ordered tests verify protection of pending readers, and update tests check regenerated boundaries and rejection of stale plans.

**Index Terms**—GPU buffer reuse, zero-copy IPC, buffer lifetime, alias analysis, autonomous driving.

## I. INTRODUCTION

Consider a driving stack built on Autoware or another framework based on ROS 2, the Robot Operating System [1], [2]. A camera–LiDAR fusion node publishes a bird’s-eye-view (BEV) feature map, a top-down grid of features around the vehicle. Separate processes for 3D detection, tracking, motion planning, and data recording read it on their own schedules (Fig. 1a). We call these processes *recipients* and the components within them that access the map *readers*. For example, BEVFusion feeds one BEV map to both a detection and a map-segmentation head [3]. Our evaluated map is a  $1\times 512\times 180\times 180$  fp32 tensor of 63.3 MiB [3]. Private copies for four recipients add 256 MiB of device memory (64 MiB each after allocator rounding), a cost that grows with the number of recipients and resident frames. This motivates sharing on automotive systems-on-chip whose GPU shares DRAM with the rest of the stack [4]. Zero-copy middleware lets recipients map the sender’s buffer [5], [6], and recent ROS 2 facilities extend this to GPU memory [7], [8]. The producer must then decide when it may overwrite that buffer, which we call a *slot*, with the next frame.

Two common release rules leave a gap (Section II). *CUDA stream-only release*, shortened to stream-only in the figures, reuses the slot once all GPU work submitted so far has finished.

A queued recipient that has not submitted its reads can then silently compute on the next frame: a detector reports objects seen in frame  $k+1$  under the timestamp of frame  $k$ . At 20 Hz, a vehicle driving at 20 m/s moves a full meter between the two. *Full retention* holds the slot until every result is ready, as reference-counted zero-copy transports do when recipients keep their references until done [5], [6]. With a one-slot pool, a new frame is dropped while that slot is held. In our 20 Hz trace, the hold barely exceeds the frame period, so full retention drops every other frame and leaves the GPU idle between accepted frames (Fig. 2).

**Key insight.** A shared tensor, the *source*, has two lifetimes (Fig. 1b). Its *storage lifetime* ends when the last operation that reads it completes. Its *provenance lifetime* lasts until every result is delivered, because those results still identify the frame they came from. The two can differ widely: in the TransFusion detection head that reads our BEV map, only the first of 287 operations reads it. Later operations use intermediate results in private storage, so they can continue after the shared input is overwritten. As with a returned library book still cited in an essay, the identity outlives the need to hold the original. Each recipient therefore holds a *borrow*, permission to read the slot, only until its last source read completes.

Full retention and private copying fit the same registration rule: protect each intended recipient before it starts, then end its borrow at output completion or copy completion. Early reuse instead ends the borrow at the last source read. These are alternative release points; copying can be preferable when that read comes late.

Ending the borrow at the last read is safe only if three questions have answers. (1) Which operation reads last? *Views*, tensors that share the input’s storage, extend its reads, and one model edit can move this *read boundary* from the first operation to the second-to-last (Fig. 3). (2) Who may still read? A reader that has not yet submitted GPU work leaves nothing to wait on. (3) Is the analyzed program the one that runs? A boundary derived for one model version can be wrong for the next.

We present BEV-in-Memory (BIM), a cross-process borrowing contract motivated by BEV sharing, and ReadSeal, its mechanism for answering these questions. BIM follows the per-consumer release pattern used by BufferQueue and NvSciStream [9], [10]. It registers every recipient’s borrow when a frame is published, before any recipient starts (Section III). ReadSeal derives each read boundary by alias

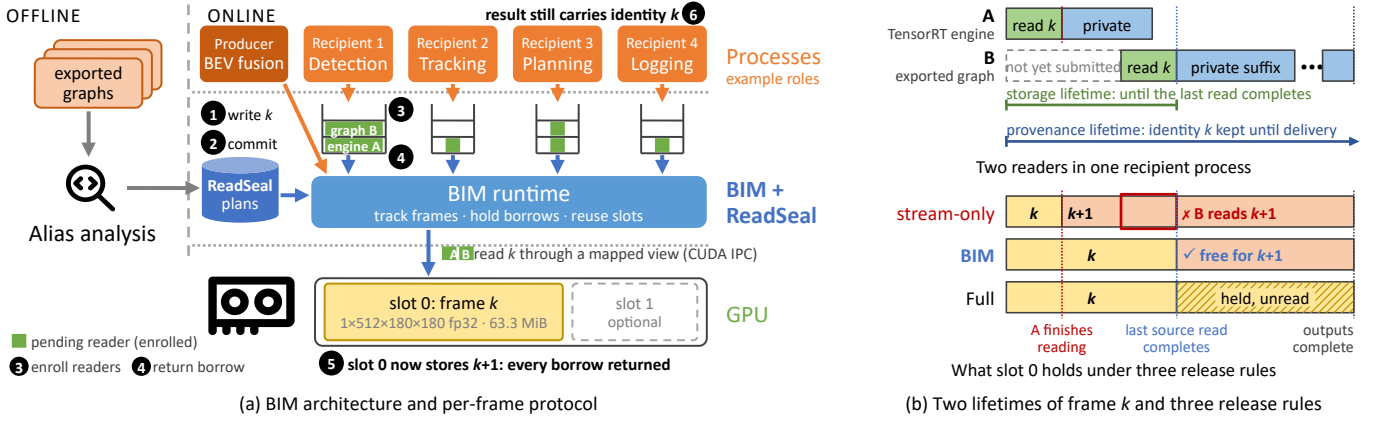


Fig. 1. **BIM at a glance.** (a) Offline, ReadSeal analyzes each exported model graph into a plan. Online, the producer writes frame  $k$  into a slot and commits it (1, 2); each recipient enrolls its readers before they submit GPU work (3) and returns its borrow once their source reads complete (4); the producer may overwrite the slot after every borrow is returned (5), while results still carry identity  $k$  (6). (b) Time runs left to right; TensorRT engine A and exported graph B are the readers of the last recipient holding a borrow. Stream-only release lets  $k+1$  overwrite the slot before the graph reads it; BIM frees it at the last source read.

analysis of the model graph, enrolls the components that may read it, and binds the plan to the code, model state, and input layout that execute (Section IV). The contribution is to connect this registration of future GPU readers to a derived, checked release point, whose placement existing release-fence interfaces leave to the application.

Automating the boundary matters because the point an engineer would place by hand must remain correct after model changes. A new late read can move it from the first operation to the second-to-last. ReadSeal recomputes that point from each model version and rejects a plan that no longer matches execution. We evaluate delivery under load and boundary maintenance separately, using hand-placed boundaries to isolate the benefit of early reuse from its automation. Our contributions are:

- **The two-lifetime view of a shared GPU tensor** (Section II). It explains how to reuse storage while retaining the frame identity of results, including when recipients have not yet submitted their reads.
- **Automatic GPU release** (Sections III–IV). Building on per-consumer release, BIM and ReadSeal protect readers before submission, derive their last-read boundaries, and reject plans that no longer match execution.
- **An evaluation of the reuse choices** (Section V), comparing early reuse, full retention, extra slots, and copying under load, with model-update tests of automatically maintained boundaries.

At 30 Hz, one BIM slot reaches nearly the same timely delivery as two slots under full retention with a tuned request cap (Section V-C), using one fewer 64 MiB source buffer.

## II. BACKGROUND AND MOTIVATION

**GPU background.** A *kernel* is a function executed on the GPU. A process places kernels in a *stream*, which executes them in order, and continues without waiting. A read ends when its kernel completes. An *event* marks a position in a stream and completes after the preceding work. PyTorch’s

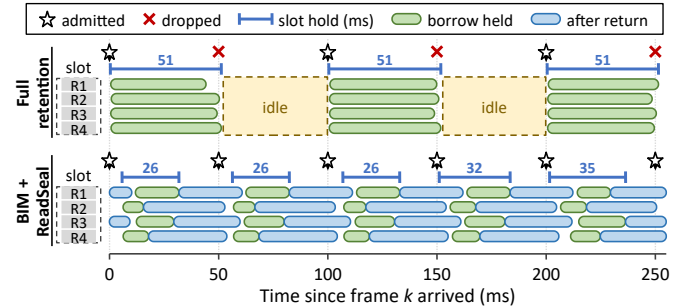


Fig. 2. **Full retention refuses frames while the GPU idles.** Measured one-slot trace at 20 Hz (repetition 0); R1–R4 are the recipients, and labels give slot hold times in ms. Full’s 51 ms hold just exceeds the 50 ms frame period, so every other frame is refused although the recipients sit idle: full retention is not work-conserving. Under BIM, each recipient’s work after its borrow return overlaps the next frame.

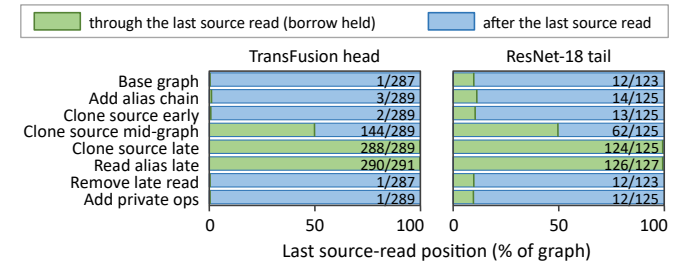


Fig. 3. **Model edits move the last source read.** Each bar is a graph snapshot; green marks the operations through the last source read, during which the recipient holds its borrow. Each snapshot applies one edit to the base graph (Remove late read undoes Clone source late); labels give the last source-reading operation/total operations. Add alias chain inserts two storage-sharing views: the operation indices shift, but the runtime release point is unchanged.

`record_stream` lets the caching allocator reuse a buffer only after the work already enqueued on the recorded streams has completed [11]. CUDA inter-process communication (IPC) lets another process map the same device buffer and wait on shared events. But an event can mark only work already

enqueued: no process can wait on reads that another has not yet submitted.

**Releasing too early fails silently.** The simplest case is a recipient still waiting in its queue. It has enqueued no GPU work, so a rule that waits only for submitted work, such as `record_stream` or a read event, returns the slot once the other recipients’ reads complete. The producer writes  $k+1$ , and the waiting recipient later computes a numerically valid result on it while its request still names  $k$ . No crash or error signals the failure, so a downstream planner has no reason to distrust the result. The same gap can open inside a recipient that combines a TensorRT engine with a framework graph. In our running example (Fig. 1b), the two *readers* are TensorRT engine A, which signals an input-consumed event after reading its input [12], and exported graph B. An adapter forwards a view to the graph without reading the source itself. Once the engine’s event completes, stream-only release returns the slot although the graph has not submitted its reads. Section V-E tests this ordering. ROS 2’s CUDA buffer backend exposes the same choice: a subscriber’s read handle records a read event when destroyed, and the block is recycled once every handle is released [13]; destroying the handle after the engine’s read is stream-only release, and holding it until the outputs complete is full retention.

**Holding the source through computation.** With one slot, full retention’s 51 ms hold exceeds the 50 ms period at 20 Hz, so every other frame is refused while the GPU idles (Fig. 2). Adding a slot absorbs some of those arrivals, but can also admit more work than the GPU finishes before the deadline. Section V-C measures this interaction with the request cap.

**Copying the source.** A private copy lets a recipient compute after the shared slot is reused. The copy itself reads the source, however, so the slot must remain protected while a recipient is queued and until its copy completes. Our clone-on-accept policy uses the same recipient registration as BIM and ends each borrow at copy completion, with extra storage for the copies.

**Maintaining a read boundary.** A view can carry a source read into later operations (Fig. 3), so a hand-placed boundary must be reviewed after model changes. A stale boundary can release too early unless execution is guarded against that change. Section V-E distinguishes changes to the actual runtime boundary from shifts in operation numbering. Table I summarizes the release choices.

### III. BIM: FRAME IDENTITY AND BUFFER REUSE

BIM separates memory, frame identity, and permission to read. The producer has a pool of  $N$  reusable GPU slots on one machine. It writes frame  $k$  into a slot and attaches an identity that every result from that frame carries. The same slot can later hold frame  $k+1$ .

A recipient process holds one borrow until all its reader components, such as engine A and graph B, have finished accessing the source. This includes readers still waiting to submit work.

TABLE I  
RELEASE OPTIONS FOR FOUR RECIPIENTS ( $K$ : REQUEST CAP). EXTRA MEMORY COUNTS SOURCE SLOTS AND INPUT COPIES. <sup>†</sup>MANUAL USES AN INDEPENDENT DIGEST-CHECKED RUNTIME; MANUAL-CHECKED USES BIM’S RUNTIME AND CHECKS. BOTH REQUIRE PLAN REFRESH AFTER A REJECTED CHANGE. <sup>‡</sup>MISSSES READERS THAT HAVE NOT SUBMITTED WORK.

| Policy                      | Extra memory | Slot reusable after | Read boundary | Wrong frame?          |
|-----------------------------|--------------|---------------------|---------------|-----------------------|
| Stream-only                 | —            | submitted reads     | stream idle   | possible <sup>‡</sup> |
| Full retention              | —            | all outputs         | —             | no                    |
| Full, 2 slots, $K=8$        | one slot     | all outputs         | —             | no                    |
| Clone-on-accept             | 256 MiB      | all copies          | fixed         | no                    |
| Manual <sup>†</sup>         | —            | all last reads      | by hand       | no                    |
| Manual-checked <sup>†</sup> | —            | all last reads      | by hand       | no                    |
| BIM + ReadSeal              | —            | all last reads      | derived       | no                    |

**Descriptors and views.** Recipients receive small descriptors over a control plane such as ROS 2 and map the resident tensor, here through PyTorch’s CUDA IPC tensor-sharing support [14]. A descriptor carries a frame identity (producer instance and sequence number), a storage locator, and an interpretation: element type, shape, strides, and a checked view range. The *view* specifies which bytes a recipient may address and how to interpret them. BIM rejects stale identities and incompatible interpretations.

**Borrow bits and admission.** A process-shared lock protects slot reservation, the frame sequence, and one borrow bit per recipient, set while that recipient may still read the slot. An arriving frame takes a slot whose bits are all clear. If none is free, the frame is dropped so the producer can keep its fixed arrival schedule; blocking would change the offered workload through backpressure. Once the frame is written, the producer *commits* it by setting every recipient’s bit before sending any descriptor. This protects even a recipient waiting in its queue. Each recipient clears its bit when it returns its borrow, and clearing the last bit makes the slot reusable without freeing or unmapping it. If frame  $k$  is committed at time  $c_k$  and recipient  $j$  returns its borrow at time  $t_{j,k}$ , the slot is held for  $H_k$ . With one slot, a frame arriving at time  $a$  is admitted only if

$$a \geq c_k + H_k, \quad H_k = \max_j t_{j,k} - c_k. \quad (1)$$

Full retention sets  $t_{j,k}$  to the time  $j$ ’s outputs complete, and ReadSeal to the time  $j$ ’s last source read completes (Section IV-B). This difference in hold time also changes which new frames can enter. An optional cap  $K$  bounds outstanding recipient requests, counted from admission to receipt at the output sink. A frame must pass both admission gates: a free slot and room under the cap.

### IV. READSEAL: CHECKED BORROWING

ReadSeal prepares a *plan* once per model version, then checks it each time a recipient processes a frame (Algorithm 1). The plan records where the graph stops reading shared storage, which components may read it, and which program version was analyzed. Sections IV-A–IV-C describe these parts in order. One execution of the plan is an *invocation*. Its recipient returns

**Algorithm 1** BIM’s commit and ReadSeal’s plan and invocation

---

```

1: procedure PUBLISH(frame  $k$ )                                ▷ producer (BIM)
2:   if no slot is free or  $K$  requests are outstanding, drop  $k$ 
3:   write  $k$  into a free slot  $s$ ; commit: set  $b_{s,j}$  for all  $j$ 
4:   send each recipient a descriptor of  $(k, s)$ 
5: procedure BUILDPLAN(graph  $G$ )                                ▷ offline, per model version
6:   compute  $Q_r$  by (2) and  $\beta$  by (3)
7:   split  $G$  at  $\beta$ ; record readers  $U$  and tokens  $T$ 
8: procedure INVOKE(plan, frame  $k$ )                                ▷ recipient  $j$ 
9:   check  $T$ ; keep identity  $k$  for every output
10:  enroll every reader in  $U$  before any of them submits GPU work
11:  submit the stages in declared order, checking  $T$  at each entry
12:  on failed check: cancel unsubmitted reads, drain, publish nothing
13:  on each completed or cancelled read: if (4) holds, clear  $b_{s,j}$ 
14:  on completed outputs: publish them with identity  $k$ 

```

---

the borrow when every planned source read has finished or was cancelled before starting.

### A. Deriving the Read Boundary

The analysis runs on each graph, a fixed-shape, functional directed acyclic graph (DAG) of operations. Of the two lifetimes, only storage needs analysis. Every result carries the frame identity attached at acceptance. Storage sharing propagates through views: a slice of the source is a view, so an operation on the slice still reads the source, whereas a convolution reads the source but writes fresh storage, so operations on its output do not. In our running example, the adapter only forwards a view; the graph receiving that view reads the source.

Formally, let  $S(v)$  be the set of shared roots whose storage a value  $v$  may alias, with  $S(r) = \{r\}$  for the source  $r$ . For an operation  $o$  with inputs  $v_i$  and output  $v_o$ , let  $\mathcal{A}_o$  index the inputs the output may alias and  $\mathcal{R}_o$  the inputs  $o$  reads. Then

$$S(v_o) = \bigcup_{i \in \mathcal{A}_o} S(v_i), \quad (2)$$

$$Q_r = \{o : \exists i \in \mathcal{R}_o, r \in S(v_i)\},$$

where  $Q_r$  is the set of operations that may read the source. With the operations numbered  $o_1, \dots, o_n$  in execution order, the read boundary is the position of the last one in  $Q_r$ ,

$$\beta = \max\{i : o_i \in Q_r\}, \quad (3)$$

so the *source-reading prefix*,  $o_1, \dots, o_\beta$ , includes all source reads. The *private suffix*,  $o_{\beta+1}, \dots, o_n$ , computes the remaining results using private intermediate storage. Fig. 3 labels each graph snapshot with  $\beta/n$ ; the base head has  $\beta = 1$  and  $n = 287$ .

Operator summaries cover 42 reviewed operator variants in ATen, PyTorch’s operator library, plus selection of a tuple element at a fixed index. Each summary counts every operand as read and treats an output as sharing storage whenever the operator’s declared signature permits aliasing with an input. Plan construction rejects unsupported mutation, opaque callbacks, and source views that escape the graph. The implementation compiles the prefix and suffix into separate executable stages. A CUDA event after the prefix then marks completion of all its source reads.

### B. Enrolling Readers Before Submission

Registration follows the two consumer levels in Section III. BIM protects each recipient process by setting its borrow bit at commit. When that process accepts a frame, ReadSeal registers, or *enrolls*, all its reader components before any of them submits GPU work. Each reader  $u$  marks its last source read with a completion event: A’s input-consumed event or the event after B’s prefix. Let  $U_j$  be the readers of recipient  $j$ . Then  $j$  returns its borrow for frame  $k$  at the first moment when

$$\forall u \in U_j : \quad done_u \vee cancelled_u, \quad (4)$$

where  $done_u$  holds once  $u$ ’s completion event has completed and  $cancelled_u$  once  $u$  has been cancelled before submission. Every enrolled reader participates in this condition. When A’s event completes, B’s read is still pending, so the bit stays set even though B has enqueued no GPU work. Returning the borrow clears  $j$ ’s bit. Once all recipients have returned their borrows, the producer can write the next frame while  $j$ ’s private suffix continues computing results labeled  $k$ .

### C. Binding Plans to Execution

At construction, ReadSeal captures the functions to run and clones model parameters and buffers once into plan-owned storage to isolate them from later caller edits, as does the manual implementation. These per-plan allocations are separate from the per-frame input copies in Table I.

ReadSeal records *binding tokens*  $T$ , identifiers and metadata for code and globals, model storage and version, input layout, and compiled stages. At acceptance and each stage entry it compares them with the values for the execution about to run,  $\hat{T}$ , and proceeds only if  $\hat{T} = T$  (Algorithm 1). A failed check cancels unsubmitted reads, drains started work, and publishes nothing. The application supplies the exported graph, each native engine’s last-input-read event, and the component and output dependencies. New operators require reviewed summaries.

### D. Correctness Conditions

The analysis must include every operation that can read the source. It assumes that operator summaries conservatively describe reads and storage sharing, kernels obey those summaries, and the graph executes in the analyzed order.

**Proposition 1.** *Under these assumptions,  $Q_r$  in Eq. (2) contains every operation that can read the shared source. No operation after the derived boundary  $\beta$  reads that source.*

*Proof sketch.* Initially  $S(r) = \{r\}$ . At each operation, the union in Eq. (2) preserves the shared roots of every input that its output may alias. Induction over graph order therefore includes every source alias in the corresponding  $S(v)$ . The read rule places every operation consuming such storage in  $Q_r$ ; by Eq. (3), none follows  $o_\beta$ .  $\square$

For the runtime, each reader’s completion event must occur after its source reads, mappings must remain valid, and identities must remain unique while referenced. Algorithm 1

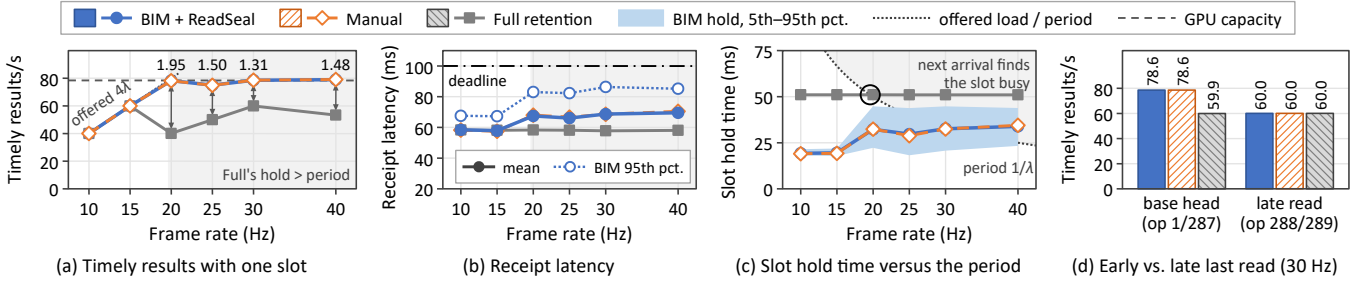


Fig. 4. **Early reuse raises one-slot delivery under load, with a latency cost.** One slot, no pipeline-wide cap  $K$ ; six runs per condition and policy. Each input frame offers four recipient results; results/s counts individual results. (a) Labels give BIM/Full ratios. (b) Mean receipt latency; BIM’s 95th percentile is computed per run and averaged. (c) Mean hold and BIM’s 5th–95th percentile; shading marks where Full’s hold exceeds the frame period. (d) Moving the last read to the graph’s end removes the gain. These runs are separate from the slot campaign in Fig. 5.

registers every reader before submission, checks the plan at execution, and returns a borrow only when Eq. (4) holds. BIM permits overwrite after all recipient bits clear. Together, these steps preserve the source until every reader finishes, while results keep the identity captured at acceptance.

## V. EVALUATION

The evaluation answers five questions in turn: when early reuse pays and what it costs in latency (Section V-B, Fig. 4); whether a second slot could do the same (Section V-C, Fig. 5); how jitter, mixed recipients, and copying change the picture (Section V-D, Fig. 6); whether BIM stays safe under reordering and model change (Section V-E); and what ReadSeal’s checks cost (Section V-F).

### A. Setup

**Scope and platform.** We evaluate cross-process reuse of saved BEV features on one NVIDIA A100 PCIe 40GB GPU. The producer, four recipients, and output sink communicate through multiprocessing queues and share storage and events through CUDA IPC. This setup measures reuse and admission independently of feature extraction; it does not run ROS 2, the BEV front end, or an embedded driving stack. The runtime uses PyTorch 2.1.2, CUDA 12.1, and TensorRT 10.3. Without NVIDIA’s Multi-Process Service (MPS), recipient kernels take turns on the GPU. Records, validators, campaign details, and analysis scripts are public.<sup>1</sup>

**Workload.** The exported graph (B) is BEVFusion’s functional TransFusion head [3], [15], with 287 operations. The producer publishes precomputed 63.3 MiB feature maps from three fixed nuScenes-mini frames [16]. The native engine (A) runs the head’s shared convolution, ReLU, and global average pooling in TensorRT; an adapter forwards a view to the graph. This controlled composition gives each recipient two readers of the same source, each with its own completion event. Four recipients run identical compositions in the rate and slot sweeps; Section V-D varies their components. A pretrained ResNet-18 tail, comprising residual stages and classifier with saved stem activations as the shared source, serves in the correctness, model-change, and cost tests.

**Policies.** All policies share the pool, workloads, queues, and admission settings. BIM uses ReadSeal, and *Full* retains the source until its outputs complete. Two hand-placed baselines isolate different costs. *Manual*, used in the rate and slot sweeps, independently registers both readers and checks code and model digests. It tests early reuse with a separately implemented runtime. *Manual-checked*, used in the jitter and mixed-recipient tests, shares BIM’s runtime and binding checks, changing only the placement of the boundary. It isolates automatic placement from execution checks. *Clone-on-accept* gives each recipient a private input copy when it accepts a request.

**Metrics.** The producer schedules frames at rate  $\lambda$ , in Hz. Each frame offers one *result*, the output bundle delivered by one recipient, to each of four recipients, giving an offered load of  $4\lambda$  results/s. A result is *timely* if it reaches the sink after device-to-host transfer and IPC, passes source-identity and reference-output checks, and arrives within 100 ms of the frame’s planned arrival. We use 100 ms as a fixed benchmark threshold to compare delivery at a common latency budget, and re-score the slot runs at 80–200 ms to show deadline sensitivity. *Receipt latency* runs from planned arrival to sink receipt; *slot hold time* runs from commit to the last borrow return. Complete-frame timely fraction (Section V-D) requires all four intended results to be correct and timely. Its denominator includes every scheduled frame, including refused frames and partial admissions.

**Queue settings.** The rate and slot sweeps permit at most two pending requests and four in flight per recipient. Optional cap  $K$  limits outstanding recipient requests across the pipeline;  $K=8$  accommodates two full groups of four requests. *No pipeline-wide cap* leaves the per-recipient queue limits in place.

**Statistics.** Rate and slot runs have 2 s of warm-up, 12 s of measured arrivals, and a complete drain. Condition and method order are randomized, with six paired repetitions per condition; brackets give pointwise 95% intervals from 20,000 paired whole-run bootstrap draws. The two campaigns in Section V-D use queues of eight, 10 s of warm-up, and 30 s of measurement, with six paired runs and pointwise 95%  $t$  intervals. Campaigns are analyzed separately. Latency summaries describe individual recipient results in the rate sweep (mean and per-run 95th percentile) and complete correct frames in the mixed-recipient study (per-run 99th percentile).

<sup>1</sup><https://github.com/assasin831/readseal-artifact/tree/ispa2026-r17>.

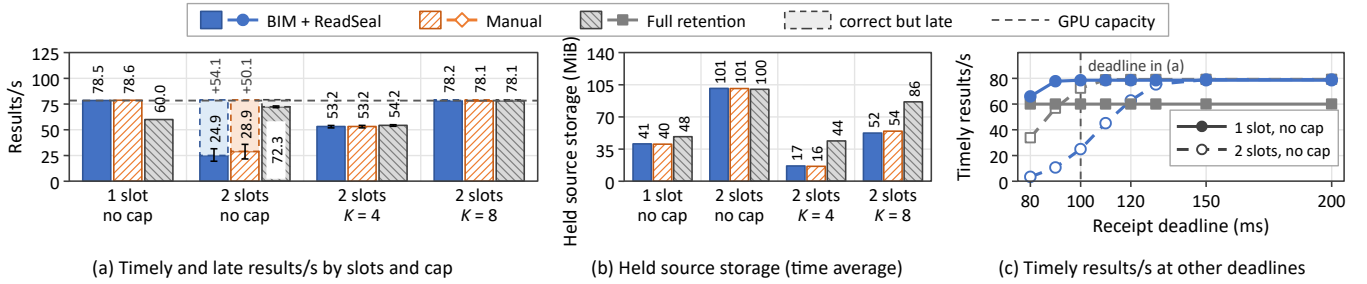


Fig. 5. **One BIM slot matches full retention with two slots and a request cap** (30 Hz; six runs per condition and policy).  $K$  counts outstanding recipient requests; eight requests are two full groups of four. (a) Solid: timely results; dashed: correct but late; whiskers: 95% intervals. (b) Time-averaged source storage held by unreturned borrows, not allocated memory. Device peaks are 5,078 MiB with one slot and 5,142 MiB with two for every policy. (c) BIM and Full re-scored at different deadlines using the same runs.

### B. Early Reuse Raises Delivery Under Load

Early reuse raises timely delivery once the frame period falls below full retention’s hold time (Fig. 4a). At 25–40 Hz, BIM delivers  $1.31\text{--}1.50\times$  as many timely results as Full. At lower rates both keep up, until the 20 Hz transition where Full’s hold just exceeds one frame period and the ratio peaks at  $1.95\times$ . BIM stays within 0.2 results/s of Manual at every rate.

**Slot hold time.** Full holds the slot for about 51 ms, spanning nearly the entire computation of the four recipients. BIM returns it after their source reads, reducing the hold to 30–34 ms (Fig. 4c). Even though each base head reads the source only in its first operation, the last recipient must wait its turn on the shared GPU before that read runs. The shorter hold lets the next frame arrive while private computation continues, filling the idle gaps in Fig. 2. Periodic admission still produces the dip at 25 Hz: after a refused frame, the producer waits until its next scheduled arrival.

**Receipt latency.** The additional admitted work queues behind other recipients on the GPU. At 20–40 Hz, BIM’s mean latency rises by 8–11 ms and its per-run 95th percentile by 15–19 ms relative to Full (Fig. 4b). The delivery lead at 30 Hz remains stable across 100–200 ms deadlines, but narrows at 80 ms because more of this queued work arrives late (Fig. 5c).

**Late source reads.** Moving the last source read to operation 288 of 289 leaves almost no private computation to overlap (Fig. 3’s late clone). BIM’s hold then reaches Full’s, and all three policies deliver 60.0 results/s at 30 Hz (Fig. 4d).

### C. One Slot Matches Two, and Two Need a Cap

Double buffering can recover the delivery lost to full retention (Fig. 5a). At 30 Hz, two-slot Full with  $K=8$  delivers 78.1 results/s, close to one-slot BIM’s 78.5. One-slot BIM therefore reaches the same measured capacity with one fewer source buffer. The device-peak difference is 64 MiB out of roughly 5 GiB in this pipeline (Fig. 5 caption). The larger pool also makes the request cap important: too small a cap limits delivery, whereas no cap allows more work to miss the deadline.

**Queueing with two slots.** One slot paces BIM’s arrivals through the hold time in Eq. (1). A second slot weakens that pacing, allowing requests to queue before the GPU can start them. Without a pipeline-wide cap, two-slot BIM delivers only

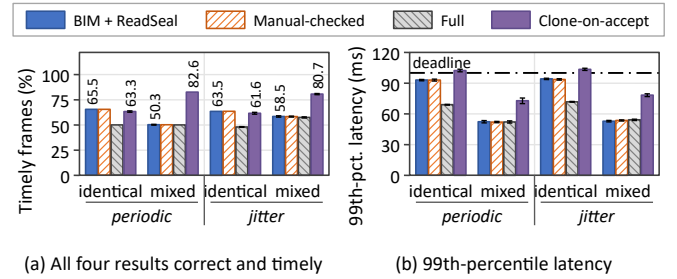


Fig. 6. **Complete-frame delivery and tail latency with jitter and mixed recipients** (30 Hz, one slot, no pipeline-wide cap; 6 runs per bar; whiskers: 95%  $t$  intervals). (a) Share of the 900 scheduled frames per run whose four results are all correct and timely; (b) per-run 99th-percentile latency of frames whose four results are all correct. *Identical*: four copies of the composition; *mixed*: four component profiles, one reading late.

24.9 timely results/s against Full’s 72.3, although its completed results remain correct. Full’s longer hold throttles arrivals, while BIM’s earlier reuse adds queueing delay at an already busy GPU. Manual shows the same drop, and a looser deadline recovers the delayed delivery (Fig. 5c).

**Request cap.** At  $K=8$ , all three policies reach similar delivery across the two-slot rate sweep. BIM still returns borrows earlier: time-averaged source storage held by outstanding borrows falls from 86.4 MiB under Full to 52.3 MiB under BIM (Fig. 5b). The allocated pool and device peak stay unchanged because returning a borrow makes a slot reusable without deallocating it.

### D. Jitter, Mixed Recipients, and Copying

The next study tests whether every intended recipient completes on time, using the complete-frame metric in Fig. 6. At 30 Hz with one slot and no pipeline-wide cap, arrivals are periodic or displaced by up to 0.8 of a period (*jitter*). Recipients are either *identical* copies of the composition or *mixed*: the native engine alone, the exported head alone, their composition, and a composition that reads the source late. Manual-checked supplies the hand-placed comparison in this study.

**Arrival pattern and recipient mix.** With identical recipients, BIM completes 65.5% of frames on time under periodic arrivals and 63.5% with jitter, about 15.5 percentage points above Full in both cases. With mixed recipients, the late reader holds

the slot longer and BIM stays less than one point above Full (Fig. 6a). Jitter raises mixed-recipient delivery from 50.3% to 58.5% for BIM and from 50.0% to 57.5% for Full. This is consistent with irregular arrival gaps allowing more frames past a hold that otherwise refuses every second periodic arrival.

**Input copying.** Clone-on-accept releases the shared input after copying, even when computation will read the private copy much later. With mixed recipients it completes 22–32 percentage points more frames on time than BIM, at a 256 MiB higher sampled device peak and 20–25 ms more 99th-percentile latency. With identical early-reading recipients it completes about two points fewer.

A separate 168-run study measures individual recipient results, rather than complete frames. At base 25 Hz, cloning delivers 77.7 timely results/s against BIM’s 75.0; when all readers read late at 30 Hz, it delivers 76.4 against 60.0. These paired gains are 2.75 [2.47, 3.03] and 16.39 [16.06, 16.72] results/s. In the late case, cloning raises the mean per-run 99th-percentile latency from 68.7 to 101.1 ms and worsens freshness. Its sampled device peak is 5,334 MiB, against BIM’s 5,078 MiB.

**Hand-placed comparison.** Across this study, including its 12 all-late runs, Manual-checked stays within 0.13 percentage points of BIM. With the execution checks held fixed, automatic boundary derivation preserves the delivery of the hand-placed version.

#### E. Safety Across Reordering and Model Changes

**Future readers.** In six controlled ordered tests, the exported graph’s outputs under stream-only release matched the replacement frame; BIM’s and Manual’s matched the admitted frame. The graph had not yet submitted its reads when the engine completed, so recording stream use did not cover those later reads [11]. Separately, in the 144 rate and late-read runs, all 109,592 results delivered by BIM, Manual, and Full carried their admitted identity and matched reference outputs.

**Boundaries under model change.** Fig. 3 shows derived boundaries for sample snapshots; ResNet’s residual connection already needs a later boundary than the head’s. Across eight snapshots per model, three inputs, and two methods, all 96 ordered-overwrite cases (540 output tensors) matched unsplit references after the regenerated boundary, and binding validation rejected all 22 attempted substitutions of plan, code, globals, model state, or input before the affected stage.

**Boundary maintenance.** We replay 16 controlled transitions: 14 graph edits, one batch-normalization folding, and a public ResNet-18 to ResNet-50 substitution. The runtime release point changes in 13 of them. Folding preserves the head’s first source read, and two alias-chain edits only shift operation indices (Fig. 3).

Binding checks reject all 16 outdated hand-placed plans, including the model substitution checked on the CPU. Execution waits for a refreshed manual plan; ReadSeal instead constructs the updated plans in 0.08–0.21 s on the CPU without source-reader or boundary annotations. In the separate mixed-recipient correctness tests, adding a late read caused Manual-

checked’s old boundary to be rejected until it was moved. All 24 update cases then gave exact outputs.

#### F. Execution Cost and Coverage

An isolated-process test times recipient acceptance to output dispatch (8 process repetitions of 50 randomized invocations per method; three inputs; base and late variants of both models). ReadSeal’s checks add 0.8–0.9 ms per invocation, mostly at acceptance and stage submission: 8% over Manual for the base head (11.8 ms against 10.9 ms) and 22–23% for the lighter ResNet tail. In the pipelines above, this cost overlaps shared GPU work; even at 10 and 15 Hz, BIM’s mean receipt latency is within 0.4 ms of Manual’s. A static survey of 14 torchvision tails with untrained weights accepts nine, with boundaries in their first 1–12 operations, and rejects five for unsupported operators.

### VI. DISCUSSION

**Choosing a release policy.** Early reuse is useful when substantial private computation follows the last source read. A single BIM slot can then admit the next frame while the previous computation continues, with ReadSeal maintaining that boundary across supported model updates. Full retention with extra slots is a simpler alternative when the extra storage and request-cap tuning are acceptable. When recipients read late, copying can release the shared slot earlier at the cost of private storage and higher tail latency. These choices use the same requirement to protect readers that have not started.

**Portability.** BIM requires cross-process storage mapping and completion events that cover each reader’s source access. ROS 2’s CUDA buffer read handles [13] and NvSciStream’s per-consumer packet release [10] provide candidate integration points. A port must implement the storage and event adapter and validate its ordering guarantees.

**Deployment.** The prototype runs declared topologies with single-stream stages; a topology change requires a new plan. A crashed recipient leaves its borrow outstanding and prevents slot reuse. Recovery requires confirmation that the recipient’s GPU work has stopped before its borrow can be removed.

### VII. RELATED WORK

**Zero-copy robotics middleware.** iceoryx and Agnocast give ROS 2 subscribers direct access to publisher-owned shared memory [5], [6]. For GPU data, NITROS keeps its zero-copy path within one process [7], and ROS 2’s `rosidl::Buffer` moves tensor bytes between processes through CUDA IPC [8]. Readers that have not yet submitted, and where to release, are left to the application.

**Release fences.** Android’s BufferQueue lets a consumer release a pooled buffer with a fence while still reading it [9], and NvSciStream returns a multicast packet to its pool once every consumer has released it [10]. BIM’s borrow bits implement this protocol; ReadSeal derives where the release fence goes, which these interfaces leave to the application.

**Asynchronous buffer protocols.** Simpson’s four-slot mechanism separates concurrent reads and writes across buffers;

Kopetz and Reisinger’s NBW protocol detects overlapping writes and also provides a multibuffer variant [17]–[19]. Cyclical asynchronous buffers let readers acquire the latest complete value without blocking the writer [20]. BIM preserves the particular frame already assigned to each recipient, including while its GPU reads are still unsubmitted. ReadSeal supplies the model-derived completion boundary for that borrow.

**Last use and safe reclamation.** Compilers and runtimes reuse a buffer after its last use, as in TVM’s static memory planning [21], and Rust’s non-lexical lifetimes end a borrow there [22]; all see every use in one program, in program order. Safe memory reclamation, such as read-copy-update (RCU) grace periods [23], reuses memory once no reader can hold it, but protects only readers that have announced themselves. In RCU terms, BIM’s commit opens every recipient’s read-side section on its behalf, and ReadSeal closes it at the derived last read. ReadSeal’s bindings, like PyTorch 2’s guards [24], reject execution whose assumptions changed.

### VIII. CONCLUSION

BIM lets shared GPU storage serve the next frame while results from the previous frame retain their identity. ReadSeal makes this possible across processes whose readers start at different times, using model analysis to maintain the release boundary. On the evaluated workloads, early source reads leave enough private computation to improve timely delivery with one slot. Extra slots and copying offer other tradeoffs when memory, queueing, or late readers dominate. Future work will integrate BIM with ROS 2 and automotive sharing backends, measure the checks on embedded cores, and support concurrent recipient execution and selective copying for late readers.

### REFERENCES

- [1] Autoware Foundation, “Autoware: Open-source software for autonomous driving,” <https://autoware.org/>, 2024.
- [2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Sci. Robot.*, vol. 7, no. 66, p. eabm6074, 2022.
- [3] Z. Liu *et al.*, “BEVFusion: Multi-task multi-sensor fusion with unified bird’s-eye view representation,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 2774–2781.
- [4] NVIDIA, “CUDA for Tegra application note (CUDA 13.0),” <https://docs.nvidia.com/cuda/archive/13.0.0/cuda-for-tegra-appnote/index.html>, 2025.
- [5] Eclipse Foundation, “Eclipse iceoryx: True zero-copy inter-process communication,” <https://github.com/eclipse-iceoryx/iceoryx>.
- [6] T. Ishikawa-Aso and S. Kato, “ROS 2 Agnoscant: Supporting unsized message types for true zero-copy publish/subscribe IPC,” in *Proc. 28th Int. Symp. Real-Time Distrib. Comput. (ISORC)*, 2025, pp. 1–10.
- [7] NVIDIA, “Isaac ROS NITROS: NVIDIA Isaac transport for ROS,” [https://nvidia-isaac-ros.github.io/repositories\\_and\\_packages/isaac\\_ros\\_nitros/isaac\\_ros\\_nitros/index.html](https://nvidia-isaac-ros.github.io/repositories_and_packages/isaac_ros_nitros/isaac_ros_nitros/index.html), 2026.
- [8] Open Robotics, “About `rosidl::Buffer` backends (ROS 2 Lyrical documentation),” <https://docs.ros.org/en/lyrical/Concepts/Intermediate/About-Buffer-Backends.html>, 2026.
- [9] Android Open Source Project, “BufferQueue and Gralloc,” <https://source.android.com/docs/core/graphics/arch-bq-gralloc>.
- [10] NVIDIA, “NvSciStream APIs (NVIDIA DRIVE OS 6.0 Linux SDK API reference),” [https://developer.nvidia.com/docs/drive-os/6.0.8/public/drive-os-linux-sdk/api\\_reference/group\\_nvsci\\_stream\\_apis.html](https://developer.nvidia.com/docs/drive-os/6.0.8/public/drive-os-linux-sdk/api_reference/group_nvsci_stream_apis.html).
- [11] PyTorch Contributors, “`torch.Tensor.record_stream` (PyTorch 2.1 documentation),” [https://pytorch.org/docs/2.1/generated/torch.Tensor.record\\_stream.html](https://pytorch.org/docs/2.1/generated/torch.Tensor.record_stream.html), 2023.
- [12] NVIDIA, “TensorRT 10.3: Execution context and input-consumed event contracts,” <https://github.com/NVIDIA/TensorRT/blob/release/10.3/include/NvInferRuntime.h>, 2024.
- [13] Open Robotics, “`cuda_buffer_backend` (ROS 2 `roslidl_buffer_backends`),” [https://github.com/ros2/roslidl\\_buffer\\_backends](https://github.com/ros2/roslidl_buffer_backends), 2026.
- [14] PyTorch Contributors, “Multiprocessing package: Sharing CUDA tensors (PyTorch documentation),” <https://docs.pytorch.org/docs/stable/multiprocessing.html>.
- [15] X. Bai *et al.*, “TransFusion: Robust LiDAR-camera fusion for 3D object detection with transformers,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 1090–1099.
- [16] H. Caesar *et al.*, “nuScenes: A multimodal dataset for autonomous driving,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 11 621–11 631.
- [17] H. R. Simpson, “Four-slot fully asynchronous communication mechanism,” *IEE Proceedings E (Computers and Digital Techniques)*, vol. 137, no. 1, pp. 17–30, 1990.
- [18] H. Kopetz and J. Reisinger, “The non-blocking write protocol NBW: A solution to a real-time synchronization problem,” in *Proc. IEEE Real-Time Systems Symposium*, 1993, pp. 131–137.
- [19] J. Rushby, “Model checking Simpson’s four-slot fully asynchronous communication mechanism,” Computer Science Laboratory, SRI International, Tech. Rep., Jul. 2002. [Online]. Available: <https://www.csl.sri.com/~rushby/papers/4slot.pdf>
- [20] G. Buttazzo, “Why real-time computing?” *Automazione e Strumentazione*, pp. 82–88, Feb. 2007. [Online]. Available: <https://retis.santannapisa.it/~giorgio/paps/2007/automazione07.pdf>
- [21] T. Chen *et al.*, “TVM: An automated end-to-end optimizing compiler for deep learning,” in *Proc. 13th USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 2018, pp. 578–594.
- [22] N. Matsakis, “RFC 2094: Non-lexical lifetimes,” <https://github.com/rust-lang/rfcs/blob/master/text/2094-nll.md>, 2017.
- [23] P. E. McKenney and J. D. Slingwine, “Read-copy update: Using execution history to solve concurrency problems,” in *Proc. Parallel Distrib. Comput. Syst. (PDCS)*, Oct. 1998, pp. 509–518.
- [24] J. Ansel *et al.*, “PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation,” in *Proc. ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, vol. 2, 2024, pp. 929–947.